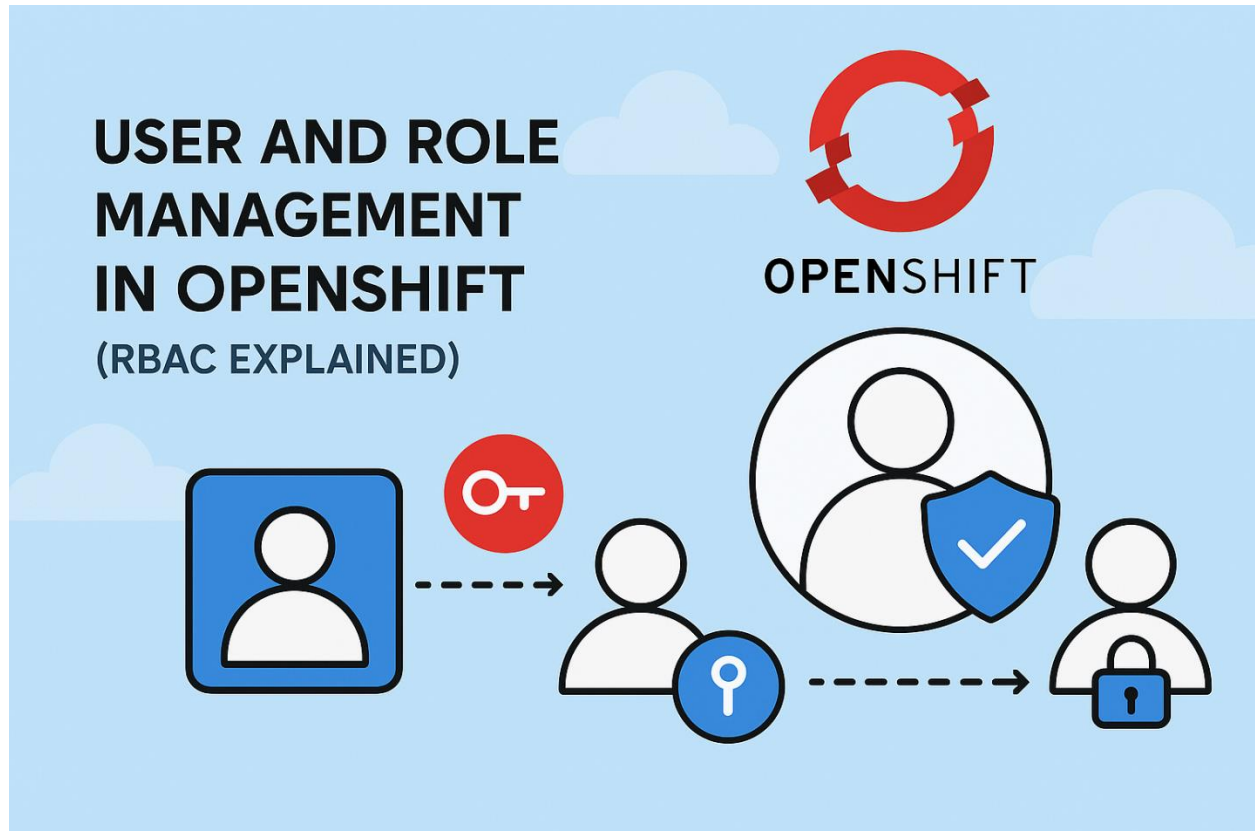


For more details, please visit <https://assistedlabs.com>

# User and Role Management in OpenShift (RBAC Explained)



## Introduction to User and Role Management in OpenShift

OpenShift, built on Kubernetes, offers a powerful and flexible platform for managing containerized applications. A critical component of managing any secure OpenShift cluster is **Role-Based Access Control (RBAC)** — the mechanism that governs **who** can perform **what actions** on **which resources**.

### What is RBAC?

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

## ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

RBAC (Role-Based Access Control) is a method of restricting access based on the roles of individual users within an enterprise. It ensures that:

- Users are granted **only the permissions they need** to perform their jobs.
- Unauthorized access is prevented.
- Administrators can **centrally control** access to resources.

### Why is RBAC Important in OpenShift?

In OpenShift, RBAC plays a central role in:

- **Enforcing multi-tenancy** and isolating workloads.
- Managing access in **developer vs administrator** scenarios.
- Protecting cluster-wide resources from unintended changes.
- Supporting **audit and compliance** requirements.

---

## Key Concepts in OpenShift RBAC

To implement RBAC, OpenShift defines four key concepts:

### 1. Users

Users represent identities that interact with the OpenShift cluster. They can be:

- **Real users** (e.g., developers, admins).
- **Service accounts** (used by applications).

OpenShift can integrate with **identity providers (IdPs)** like LDAP, OAuth (GitHub, Google), etc.

### 2. Groups

Groups are collections of users. Permissions can be assigned to a group instead of individual users to simplify access control.

Example:

```
oc adm groups new dev-team user1 user2 user3
```

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

## ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

### 3. Roles

A role defines a **set of permissions** (rules) that apply to a resource. Roles can be:

- **ClusterRole** – permissions that apply across the cluster.
- **Role** – permissions that are **namespaced** (apply only within a project/namespace).

### 4. RoleBindings

RoleBindings connect **users/groups/service accounts** to **roles**.

- **RoleBinding** – binds to a Role within a specific namespace.
- **ClusterRoleBinding** – binds to a ClusterRole cluster-wide.

---

## Creating Users, Roles, and RoleBindings

### Adding Users in OpenShift

OpenShift doesn't store users itself — instead, it delegates authentication to an identity provider (e.g., httpasswd, LDAP).

#### Example: Create user via httpasswd

```
htpasswd -c -B /etc/origin/httpasswd user1  
oc create secret generic htpass-secret --from-file=htpasswd=/etc/origin/httpasswd -n openshift-config
```

Then reference this secret in the OAuth configuration.

### Creating Roles

A Role is a set of permissions to access specific resources within a namespace.

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

#### Example: Create a Role

```
kind: Role  
apiVersion: rbac.authorization.k8s.io/v1  
metadata:  
  namespace: dev  
  name: pod-reader
```

## ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

rules:

```
- apiGroups: [""]  
  resources: ["pods"]  
  verbs: ["get", "list", "watch"]
```

### Creating RoleBindings

#### Example: Bind Role to User

```
kind: RoleBinding  
apiVersion: rbac.authorization.k8s.io/v1  
metadata:  
  name: read-pods  
  namespace: dev  
subjects:  
- kind: User  
  name: user1  
roleRef:  
  kind: Role  
  name: pod-reader  
apiGroup: rbac.authorization.k8s.io
```

---

## Cluster Roles and Cluster RoleBindings

### ClusterRole

ClusterRoles grant permissions that span across all namespaces. They are used for administrative tasks like node management, persistent volumes, etc.

#### Example: View ClusterRole

```
oc describe clusterrole view
```

### Creating a ClusterRoleBinding

#### Example: Grant admin access cluster-wide

```
oc adm policy add-cluster-role-to-user cluster-admin user1
```

This command creates a **ClusterRoleBinding** that allows user1 full administrative access to the cluster.

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

## ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

### Common Predefined ClusterRoles

| ClusterRole     | Purpose                         |
|-----------------|---------------------------------|
| cluster-admin   | Full control of the cluster     |
| admin           | Full control within a namespace |
| edit            | Edit access to most resources   |
| view            | Read-only access                |
| system:deployer | For deployment configurations   |

## Best Practices and Troubleshooting

### Best Practices for User and Role Management

1. **Follow the principle of least privilege** – give users only the permissions they need.
2. **Group users** and assign permissions to groups for scalability.
3. **Use namespaces (projects)** to isolate environments (e.g., dev, test, prod).
4. **Audit access regularly** to ensure compliance.
5. **Avoid overusing cluster-admin** privileges unless truly necessary.

### Auditing and Troubleshooting Access

To troubleshoot access issues, you can check:

```
oc auth can-i <verb> <resource> --as=<user>
```

Example:

```
oc auth can-i get pods --as=user1 -n dev
```

This tells whether user1 can access pods in the dev namespace.

### Revoking Access

To revoke access, you can remove the role binding:

```
oc delete rolebinding read-pods -n dev
```

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>

## ASSISTED LABS

For more details, please visit <https://assistedlabs.com>

Or remove the cluster role:

```
oc adm policy remove-cluster-role-from-user cluster-admin user1
```

---

## Conclusion

User and Role Management in OpenShift using RBAC is crucial for **securing resources, delegating responsibilities, and complying with enterprise access policies**. Mastering roles, bindings, and identities ensures efficient governance of your OpenShift environment.

To get the OpenShift Hands-on Experience, kindly visit: <https://assistedlabs.com>